RAG Assignment: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Pavan Reddy

Code: https://github.com/pavanreddy2480/RAG_LMA (single notebook `rag_assignment.ipynb`, with outputs saved).

Setup

The document is the FIDE Laws of Chess (25 pages). It has many exceptions, penalties and "not allowed" clauses, which suits the stress tests. I split it into 121 chunks of about 100 words with a 20-word overlap; both pipelines use these same chunks.

- Bi-encoder: `all-MiniLM-L6-v2`, normalised vectors, FAISS inner-product index (equal to cosine).
- Cross-encoder: `cross-encoder/ms-marco-MiniLM-L-6-v2`.
- LLM: `gemini-3.1-flash-lite-preview`, temperature 0, one shared prompt for both answers.
- Pipeline 1: top-10 from the bi-encoder, top-3 to the LLM. Pipeline 2: the same 10 candidates re-sorted by the cross-encoder, top-3 to the LLM.

1 Architecture Comparison

A bi-encoder encodes the query and each chunk separately into one vector each, and the score is the cosine between the two vectors. Because the chunk vectors do not depend on the query, they are computed once and stored in an index. The price is that a whole chunk is squeezed into a single vector before the query is seen.

A cross-encoder takes the query and the chunk together as one input, so self-attention runs across all query tokens and chunk tokens jointly, and it outputs one relevance score. It can therefore match specific words in the query to specific words in the chunk (for example "always" in the query against "cannot win... draw" in the chunk). Nothing can be precomputed, because the score depends on the query.

2 Qualitative and Rank Shift Analysis

For each query I marked the "gold" chunk by a phrase from the clause that really answers it, found by reading the rulebook before running the models. Table 1 gives the rank of that chunk before and after re-ranking.

Failure query 1 (extra minutes). Chunk 63, which holds the exact sentence "if the arbiter has rejected the claim, the opponent shall be awarded two extra minutes", moved from rank 4 to rank 1. The baseline top-3 was chunks 62, 61, 59 and the re-ranked top-3 was 63, 62, 61. Both answers were correct ("two extra minutes"), because chunk 62 contains a similar clause about postponed decisions. So the rank improved but the baseline did not fail here.

Query	Gold chunk	Bi-enc. rank	CE rank
Extra minutes when the arbiter rejects a draw claim (failure query 1)	63	4	1
If my opponent's time runs out, do I always win? (failure query 2, exception)	66 (and 65)	10	1
How long can a player be absent before the game is forfeited? (synonym / keyword trap)	33 (and 32)	5	9
Is a player allowed to withdraw a draw claim? (negation)	59	1	1
Is it prohibited to write the moves in advance? (negation)	47	1	1
What happens on a third illegal move? (control)	45	1	1
Rules about mobile phones	65, 72	1	1

Table 1: Rank of the gold chunk among the 10 candidates. The best-ranked gold chunk is shown where more than one qualifies.

Failure query 2 (exception). The chunk with the exception sat at rank 10 and was not given to the LLM in the baseline. The cross-encoder moved it to rank 1. Answer 1 was *"Not found in the provided context"* (the prompt tells the model to say this when the context lacks the answer, so it refused instead of guessing). Answer 2 was: "If the opponent cannot win the game by any series of legal moves, the score shall be a draw." This is the clearest case where the critical chunk sitting outside the top-3 made the baseline omit the essential clause. It is the one query where the final answer changed for the better.

Keyword trap. Re-ranking did not help here. The clause on default time (chunk 33) moved from rank 5 to 9. The query uses "absent" and "forfeited", while the rule says "arrives late" and "default time". Both pipelines retrieved the adjourned-game rule ("more than one hour late") and both answers described that rule. I think the MS-MARCO cross-encoder preferred chunks with words like "late" and "loses", but I did not test this further.

Negation queries. The bi-encoder already ranked the right chunk first for both "not allowed to withdraw" and "prohibited to write in advance", and both answers were correct and nearly identical. So on this document I could not show a negation failure of the baseline.

Other observations. For the mobile-phone query the gold chunks stayed at ranks 1 and 2, but the third chunk changed: the baseline kept chunk 66 (which has the "unless the opponent cannot win, the score is a draw" clause) and the re-ranker replaced it with chunk 2. Answer 1 therefore included the draw exception and Answer 2 did not, so here the baseline answer was slightly more complete. Re-ranking is not always better.

Overall: in 2 of 7 queries the gold chunk was promoted into the top-3 (ranks 4→1 and 10→1). Only in one of these did the answer change. In one query re-ranking made things worse, and in the rest nothing changed. With 7 queries on one document this is an illustration, not a benchmark.

3 Latency Profiling

Retrieval timings are means over 7 queries with 10 repeats each, on a laptop, with 10 candidates. The LLM time is one uncached API call, since it is identical for both pipelines and the free API quota is small.

Re-ranking makes retrieval about 6.5 times slower, but the absolute cost is about 27 ms, which is small next to the LLM call.

Stage	Time
Pipeline 1: bi-encoder retrieval (embed query + FAISS top-10)	5.0 ms
Re-ranking step (10 cross-encoder pairs)	27.4 ms
Pipeline 2 retrieval total (bi-encoder + cross-encoder)	32.4 ms (6.5× baseline)
LLM generation (same in both pipelines)	about 1.1 s
Cross-encoder on all 121 chunks, one query (for comparison)	398 ms

Table 2: Runtime of the two pipelines.

4 Scaling Question

A cross-encoder must run a full transformer pass for every (query, chunk) pair, and the result cannot be cached because it depends on the query. Running it over the whole collection costs a number of passes proportional to the number of chunks for every query. Here 121 chunks already cost 398 ms, about 15 times the cost of scoring 10 chunks, and a collection of millions of chunks would take minutes per query.

A bi-encoder avoids this: chunk vectors are computed once offline, and a query costs one encoding plus a fast nearest-neighbour search. So the first stage uses the cheap model to get a short list with good recall, and the expensive cross-encoder only refines that short list. The limit of this design is that if the right chunk is not in the top-10, the re-ranker cannot recover it.